Bài 1. Tám quân hậu.

Cho trước một bàn cờ vua kích thước $n \times n$. Hãy in ra mọi phương án đặt n quân hậu sao cho vào bàn cờ sao cho không có 2 quân hậu bất kỳ nào khống chế nhau.



Ví dụ một cách đặt quân hậu trên bàn cờ 8×8

INPUT: Đọc từ file **QUEEN.INP** dòng thứ nhất ghi số nguyên dương $n \leq 12$

OUTPUT: Ghi ra file **QUEEN.OUT** gồm nhiều dòng:

- Mỗi dòng chứa n số $x_1, x_2, \ldots, x_n$ thể hiện một phương án đặt n quân hậu trên bàn cờ trong đó $x_i = j$ tức là quân hậu được đặt tại cột i dòng j.

VD:

QUEEN.INP	QUEEN.OUT
4	2 4 1 3 3 1 4 2

- **Hướng dẫn giải:**

Đây là một bài toán kinh điển, tiêu biểu của phương pháp duyệt toàn bộ dùng quay lui.

Ý tưởng giải quyết bài toán rất đơn giản: giả sử đã đặt được (k – 1) quân hậu, xét xem có thể đặt được quân hậu thứ k không, nếu có thể đặt được quân hậu thứ k thì xét tiếp quân hậu thứ (k + 1), còn nếu không thể đặt được quân hậu thứ k thì quay lại điều chỉnh quân hậu thứ (k – 1) tới vị trí mới, để có thể đặt được quân hậu thứ k.

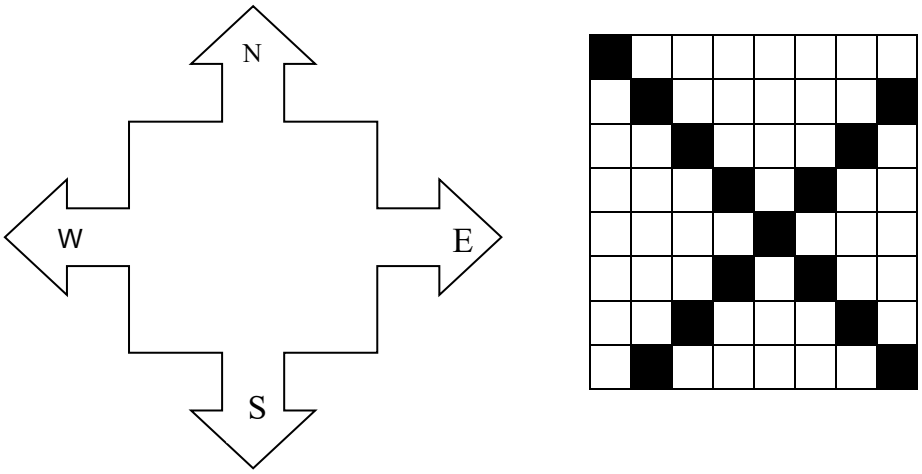
Ý tưởng trên có thể được thể hiện như sau:

Rõ ràng n quân hậu sẽ được đặt mỗi con một hàng vì hậu ăn được ngang, ta gọi quân hậu sẽ đặt ở hàng 1 là quân hậu 1, quân hậu đặt ở hàng 2 là quân hậu 2...quân hậu ở hàng n là quân hậu n. Vậy một nghiệm của bài toán sẽ được biết khi ta tìm ra được vị trí cột của những quân hậu.

Nếu ta định hướng Đông(phải), Tây(trái), Nam(dưới), Bắc(trên) thì ta nhận thấy rằng:

+ Một đường chéo theo hướng đông bắc – tây nam(ĐB-TN) bất kì sẽ đi qua một số ô, các ô đó có tính chất: hàng – cột = C(const). Với mỗi đường chéo ĐB-TN ta có một hằng số C và với 1 hằng số C: 2 <= C <=2n xác định duy nhất một đường chéo ĐB-TN vì vậy ta có thể đánh chỉ số cho các đường chéo ĐB-TN từ 2 đến 2n.

+ Một đường chéo theo hướng đông nam - tây bắc (ĐN-TB) bất kì sẽ đi qua một số ô, các ô đó có tính chất: hàng + cột=C(const). Với mỗi đường chéo ĐN-TB ta có một hằng số C và với 1 hằng số C: 1-n<=C<=n-1 xác định duy nhất một đường chéo ĐN-TB vì vậy ta có thể đánh chỉ số cho các đường chéo ĐN-TB từ 1-n đến n-1.



Hình 2: Đường chéo ĐB-TN mang chỉ số 10 và đường chéo ĐN-TB mang chỉ số 0 và ô chung (5,5)

Cài đặt:

Ta có 3 mảng logic để đánh dấu:

Mảng a[1..n], a_i=true nếu như cột i còn tự do, a_i=false nếu như cột i đã bị một quân hậu khống chế.

Mảng b[2..2n], b_i=true nếu như đường chéo ĐB-TN thứ i còn tự do, b_i=false nếu như đường chéo đó đã bị một quân hậu khống chế.

Mảng c[1-n..n-1], c_i=true nếu như đường chéo ĐN-TB thứ i còn tự do, c_i=false nếu như đường chéo đó đã bị một quân hậu khống chế.

Ban đầu cả 3 mảng đánh dấu đều mang giá trị true. (các cột và đường chéo đều tự do)

Thuật toán quay lui: xét tất cả các cột, thử đặt quân hậu 1 vào một cột, với mỗi cách đặt như vậy xét tất cả các cách đặt quân hậu 2 không bị quân hậu 1 ăn, lại thử một cách đặt và xét tiếp các cách đặt quân hậu 3... mỗi cách đặt được đến quân hậu n cho ta một nghiệm.

+ Khi ta chọn vi trí cột j cho quân hậu thứ i, thì ta phải chon ô(i,j) không bị các quân hậu đặt trước đó ăn, tức là phải chon cột j còn tự do, đường chéo ĐB-TN(i,j) cond tự do, đường chéo ĐN-TB(i-j) còn tự do. Điều này có thể kiểm tra (a_j=b_{i+j}=c_{i-j}=true).

+ Khi thử đặt được quân hậu thứ i vào cột j nếu đó là quân hậu cuối cùng (i=n) thì ta có một nghiệm nếu không:

- Trước khi gọi đệ quy tìm cách đặt quân hậu thứ i+1, ta đánh dấu cột và 2 đường chéo bị quân hậu vừa đặt khống chế (a_j=b_{i+j}=c_{i-j}:= false) để các lần gọi đệ quy tiếp sau chọn cách đặt các quân hậu kế tiếp sẽ không chọn vào những ô nằm trên cột j và đường chéo này nữa.

- Sau khi gọi đệ quy tìm cách đặt quân hậu thứ i+1 , có nghĩa là sắp tới ta lại thử cách đặt khác cho quân hậu thứ i, ta bỏ đánh dấu cột và 2 đường chéo bị quân hậu vừa thử đặt khống chế (a_j=b_{i+j}=c_{i-j}:=true) tức là cột và 2 đường chéo đó lại thành tự do, bởi khi ta đặt quân hậu sang vị trí khác rồi thì cột và 2 đường chéo đó hoàn toàn có thể gán cho một quân hậu khác.

}

Bài 2. Cái túi.

Một tên trộm lọt được vào một gia đình. Hắn mang theo một cái túi có thể tích M để đựng đồ ăn trộm. Trong nhà có N vật dụng ($N \leq 20$), vật thứ i có trọng lượng là $W_i \leq 100$ và giá trị là $V_i \leq 100$. Vậy tên trộm phải chọn những vật nào để lấy đi mà tổng giá trị của các vật đó là lớn nhất.

Input: file văn bản BAG.INP

- Dòng 1: Chứa hai số N, M cách nhau một dấu cách
- N dòng tiếp theo mỗi dòng i ghi hai số Wi và Vi cách nhau một dấu cách

Output: file văn bản BAG.OUT

- Dòng 1: Ghi tổng giá trị lớn nhất mà tên trộm có thể lấy được
- Dòng 2: Ghi chỉ số những vật dụng bị lấy đi.

Ví dụ:

BAG.INP	BAG.OUT
5 11 3 3 4 4 5 4 9 10 4 4	11 1 2 5

- **Hướng dẫn giải:**

  Xây dựng x[1..N] là phương án chọn đồ vật:

  - Với x[i] = 1 tức là chọn đồ vật thứ i
  - Với x[i] = 0 tức là không chọn đồ vật thứ i.

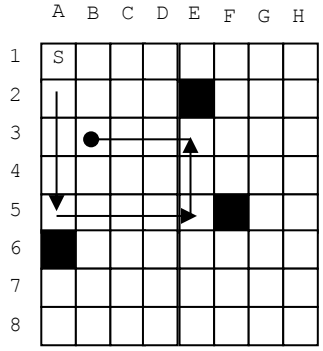
  Như vậy ý tưởng là sinh ra mọi dãy nhị phân độ dài N lưu vào mảng x[1..N], với mỗi dãy nhị phân như vậy ta thử xem các đồ vật tương ứng ở vị trí của dãy nhị phân có thể cho vừa vào túi không, nếu có thì hãy tính tổng giá trị các đồ vật và lựa chọn tổng giá trị tối ưu.

  Với test ví dụ của đề bài: ta có thể sinh được 1 dãy nhị phân độ dài 5 là 0 1 1 0 1. Tức là có thể sinh chọn các đồ vật thứ 2, 3 và 5.

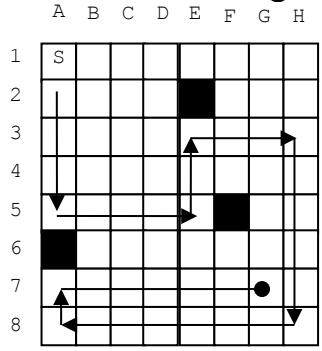
  Có thể dùng phương pháp quay lui để giải quyết bài toán dễ dàng.

Bài 3. Đường đi của con rắn

Một con rắn sống trên một lưới ô vuông 8 x 8 có chứa một vài vật cản (là các ô mang màu đen trong hình vẽ dưới đây). Con rắn di chuyển theo đường thẳng (khi còn có thể di chuyển được). Khi nó gặp vật cản hay gặp biên của lưới hình vuông, nó sẽ quay sang bên phải hay bên trái của nó. Nếu nó đối diện với ô đã đi qua, nó sẽ dừng hẳn.



Câu A: Đường đi khi rắn luôn quay bên trái (13 ô)



Câu B: Đường rắn đi với hành trình xa nhất.

Con rắn luôn bắt đầu tại ô A1 và luôn bắt đầu việc di chuyển bằng cách đi hướng xuống dưới. Vị trí của các vật cản (tức là các ô đen) được cho biết trước.

Yêu cầu:

Tìm số ô mà con rắn đi qua cho đến khi không đi được nữa và thoả mãn:

a) Con rắn luôn đi về phía bên trái của nó

b) Con rắn đi sao cho số ô đi qua là lớn nhất.

Ví dụ:

Hình vẽ trên cho hình ảnh một lưoiứ ô vuông với ba ô đen (tại các vị trí A6, E2 và F5). Đường đi thoả mãn câu a) là: đi từ A1 đến A5, quay sang trái đến E5. Từ E5 quay trái đến E3, quay trái đi thẳng và dừng tại ô B3. Con rắn đã đi qua 13 ô.

Đường đi thoả mãn câu b) được cho bởi lưới ô vuông bên phải. Đường đi nói trên đi qua 32 ô.

Dữ liệu vào: Ghi trên file SNAIL.IN gồm n+1 dòng:

- Dòng đầu tiên chi số nguyên dương n (1≤n≤32) chi số lượng các ô đen
- Trên n dòng tiếp theo, mỗi dòng ghi toạ độ một ô đen dưới dạng Xn, trong đó X (là các chữ cái Latinh hoa từ A đến Z) chỉ cột và n (1≤n≤8) chỉ dòng.

Chú ý rằng không có ô đen tại các ô A1 và A2

Dữ liệu ra: Ghi ra file SNAIL.OUT gồm 2 dòng

- Dòng đầu ghi số lượng ô đi qua thoả mãn câu a)
- Dòng thứ hai ghi số lượng ô đi qua thoả mãn câu b)

Ví dụ:

SNAIL.IN	SNAIL.OUT
3 A6 E2 F5	13 32

- **Hướng dẫn giải:**

Công việc đầu tiên là phải tạo ra được ma trận lưới 8 x 8, dựa vào input sẽ đánh dấu vị trí của các vật cản bằng 1, các vị trí còn lại mang giá trị 0, bao xung quanh ma trận là các giá trị = 1. Việc tạo ra ma trận này là không khó.

Dùng các mảng sau để điều chỉnh hướng đi của con rắn:

tx[1..4] = (0, -1, 0, 1)

ty[1..4] = (-1, 0, 1, 0)

Hai mảng sau đây điều chỉnh chiều quay trái hoặc phải của con rắn:

left[1..4] = (4, 1, 2, 3)

right[1..4] = (2, 3, 4, 1)

Có thể giải thích hai mảng ***left*** và ***right*** như sau: ví dụ left[1] = 4, right[1] = 2 tức là khi con đang đi theo hướng số 1 là hướng tx[1] = 0 và ty[1] = -1 tức là con rắn đang đi sang trái, thì hướng quay bên trái là hướng số 4 (left[1] = 4, tx[4], ty[4]) và hướng quay bên phải là hướng số 2 (right[1] = 2, tx[2], ty[2]).

Bài 4. Tìm đường đi trong mê cung

Mê cung là một ma trận A có M dòng và N cột, chỉ chứa 0 hoặc 1, trong đó 0 nghĩa là có đường đi và 1 nghĩa là không có đường đi. Một người đứng tại vị trí (u,v) có thể đi đến ô kề cạnh với ô đang đứng nếu có đường đi, hãy chỉ ra cho người đó đường đi ngắn nhất để thoát ra khỏi mê cung. Người đó ra khỏi mê cung khi đứng ở một trong 4 cạnh của mê cung.

Dữ liệu vào: Bởi file MAZE.INP gồm:

- Dòng đầu tiên chứa hai số M và N
- M dòng tiếp theo mỗi dòng ghi N số 0 hoặc 1
- Dòng cuối cùng ghi hai số U và V

Dữ liệu ra: Ghi ra file MAZE.OUT gồm:

Nếu không có đường đi ra khỏi mê cung thì ghi -1.

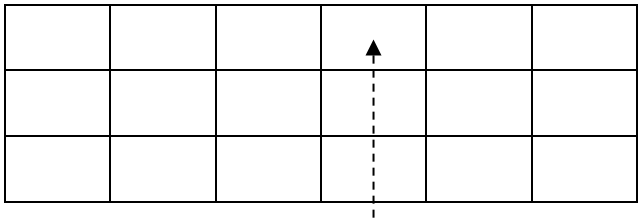
Nếu có đường đi thì:

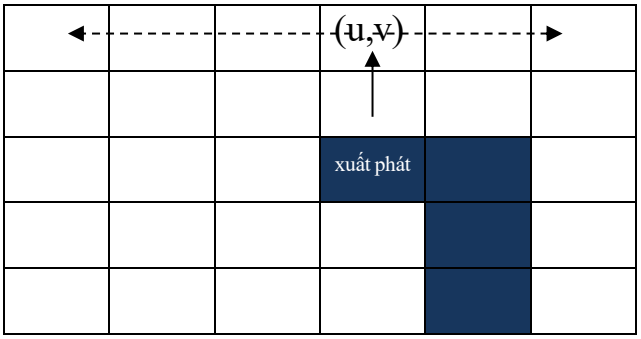
- Dòng đầu tiên ghi ra số "d" là số bước đi ngắn nhất để thoát ra khỏi mê cung
- d dòng tiếp theo mỗi dòng ghi hai số (i,j) là tọa độ các ô trong hành trình thoát ra khỏi mê cung.

Ví dụ

MAZE.INP	MAZE.OUT
5 6 1 0 1 0 1 0 0 0 1 0 0 1 1 0 0 0 0 1 1 0 0 1 0 1 1 1 1 0 1 0 3 2	2 2,2 1,2

- Hướng dẫn giải:





Tại hình vẽ, giả sử đã tìm được 1 đường đi (được tô màu xám). Sau đó quay lui để tìm đường đi ngắn hơn, chúng ta sẽ đặt cận để loại bỏ tất cả các đường đi xa hơn đường đi cũ.

Giả sử chúng ta đang đi theo hướng mũi tên màu đen đến ô (u,v). Tại đây có thể đi theo 3 hướng mũi tên có nét đứt. Nếu đi theo hướng mũi tên có nét đứt thì sẽ đến mép của bảng nhanh nhất theo mũi tên đó, ta sẽ chọn hướng mũi tên ngắn nhất để đặt cận là: **d + minc < dmin**

Trong đó

- d là độ dài đường đi từ ô xuất phát tới (u,v)
- minc là khoảng cách nhỏ nhất từ ô (u,v) đến mép bảng theo 3 mũi tên nét đứt
- dmin là độ dài đường đi ngắn nhất tìm được ở những phương án trước.

Coi hướng lên là hướng 0, xuống là 1, trái là 2 và phải là 3.

tx[0..3] = (-1,0,1,0)

ty[0..3] =(0,1,0,-1)

Bài 5. Xếp valy

Một valy có thể chứa tối đa W đơn vị trọng lượng. Có N đồ vật, số lượng mỗi loại không hạn chế. Loại đồ vật thứ i có trọng lượng là Ai và có giá trị là Ci. Hỏi nên chọn những đồ vật nào và số lượng bao nhiêu để xếp vào valy sao cho:

- Tổng trọng lượng của các đồ vật không vượt quá giới hạn W của valy
- Tổng giá trị của đồ vật là lớn nhất.

- Input: Đọc từ file **VALY.INP**
  - Dòng 1 chứa N và M ($1 \le N \le 40$, $1 \le M \le 100$)
  - N dòng tiếp theo, mỗi dòng chứa hai số A[i] và C[i].
- Output: Ghi ra file VALY.OUT

- Dòng đầu là giá trị lớn nhất

Ví dụ:

VALY.INP	VALY.OUT
4 100 50 50 19 20 80 90 21 25	110

- **Hướng dẫn giải**:

Với mỗi loại đồ vật i, gọi T_i là giá trị riêng của nó: $T_i = C_i / A_i$. Sau đó sắp các loại đồ vật theo thứ tự giảm dần của T_i.

Giả sử đã chọn được một số đồ vật trong i đồ vật đầu tiên. Điều kiện để chọn k vật của đồ vật thứ i + 1 là:

$$S + T[i+1]*(M - W) > Smax$$

Trong đó:

- S là tổng giá trị của các vật đồ vật đang có trong valy
- W là tổng trọng lượng của các đồ vật đang có trong valy
- Smax là tổng giá trị valy của một các xếp đã tìm được nhưng chưa tối ưu. Có thể khởi tạo Smax ban đầu bằng 0.